

# Guía de Actividades Práctico-Experimentales Nro. 002

## 1. Datos Generales

|                                              |                                                                                                                                                         |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Asignatura</b>                            | Desarrollo Basado en Plataformas                                                                                                                        |
| <b>Ciclo</b>                                 | 5 A                                                                                                                                                     |
| <b>Unidad</b>                                | 1                                                                                                                                                       |
| <b>Resultado de aprendizaje de la unidad</b> | Discute cómo los estándares Web influyen en el desarrollo de software, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad. |
| <b>Práctica Nro.</b>                         | 003                                                                                                                                                     |
| <b>Nombre del Docente</b>                    | Edison Leonardo Coronel Romero                                                                                                                          |
| <b>Fecha</b>                                 | viernes 17 de octubre                                                                                                                                   |
| <b>Horario</b>                               | 07h30 – 10h30                                                                                                                                           |
| <b>Lugar</b>                                 | Aula 232                                                                                                                                                |
| <b>Tiempo planificado en el Sílabo</b>       | 3 horas                                                                                                                                                 |
| <b>Integrantes:</b>                          | ANDY APOLO VALLE, Justin Guitiérrez, Brandon Medina                                                                                                     |

### Propósito

El estudiante consolida lo aprendido sobre la arquitectura de microservicios, su integración mediante API REST, y el uso de herramientas colaborativas para la gestión y documentación técnica del proyecto.

Como resultado, elaborará un documento técnico que refleje las decisiones de diseño, herramientas utilizadas y el impacto de los microservicios en la escalabilidad y mantenibilidad del sistema.

### Actividad

Actividad: Documentar el desarrollo de microservicios y su integración mediante herramientas colaborativas y APIs.

El estudiante deberá elaborar un informe técnico ilustrado, que incluya los siguientes apartados:

#### 1. Resumen del microservicio creado

- Nombre del microservicio, propósito y relación con el sistema principal.

El propósito fundamental de este microservicio es centralizar y desacoplar por completo la lógica de envío de notificaciones del resto de la aplicación. Su única responsabilidad es recibir una

solicitud con la información necesaria (destinatario, asunto y cuerpo del mensaje) y gestionar todo el proceso de comunicación con el proveedor externo, en este caso, la API de Gmail.

Funciona como un departamento especializado e independiente: el sistema principal no necesita saber *cómo* se envía un correo, ni manejar las credenciales de la API de Google, ni gestionar posibles fallos en la comunicación. Simplemente le entrega una orden y confía en que el microservicio cumplirá la tarea. Esto aumenta la modularidad y la mantenibilidad del sistema general.

Funciona de la siguiente manera:

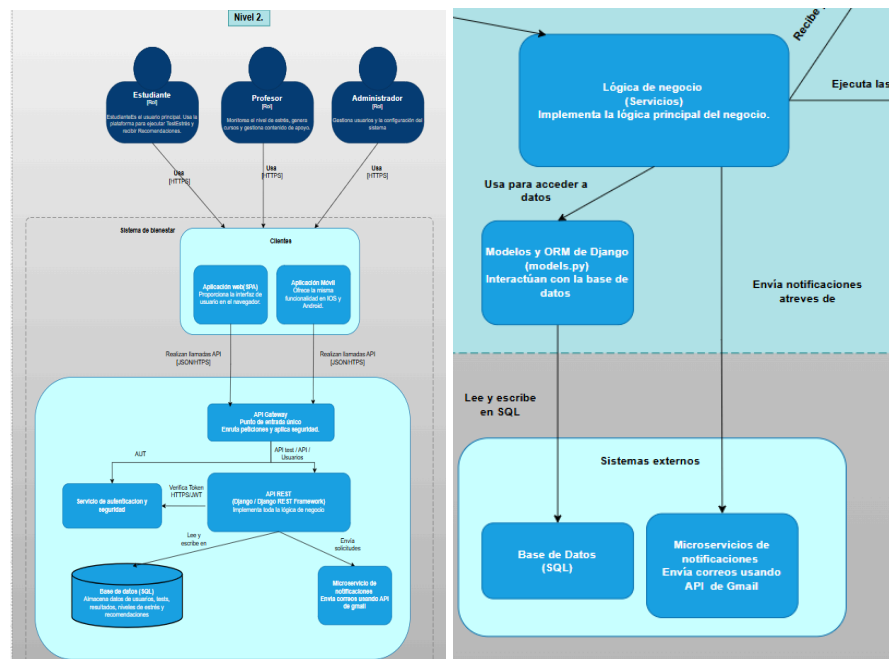
**Evento Desencadenante:** Una acción dentro del sistema principal, como un estudiante finalizando un test o un profesor generando una recomendación, activa la necesidad de enviar una notificación.

**Llamada a la API:** El sistema principal no llama directamente al microservicio. En su lugar, realiza una petición HTTP POST a un endpoint específico en el API Gateway (ej. [/api/notificaciones/enviar-correo](#)).

**Enrutamiento:** El API Gateway recibe esta petición y la redirige de forma segura al Microservicio de Notificaciones.

**Procesamiento Independiente:** Una vez que el microservicio recibe la solicitud, el sistema principal da por finalizada su tarea y puede seguir atendiendo otras peticiones del usuario, mejorando el rendimiento y la experiencia. El microservicio se encarga del resto de forma autónoma.

- Diagrama simple de su estructura (puede usar Mermaid, Draw.io o PlantUML).



## 2. Integración mediante APIs

- **Descripción del método de comunicación (REST, API Gateway o endpoints compartidos).**

Para tu arquitectura de microservicios, el método de comunicación más robusto y escalable es una combinación de **APIs REST** actuando detrás de un **API Gateway**.

- **API REST (El Lenguaje):** Cada microservicio (Auth, Cursos, Tests) expondrá su funcionalidad a través de endpoints RESTful. Esto significa que usarán los métodos HTTP estándar y se comunicarán usando JSON. Es el "lenguaje" estándar que todos los servicios entienden.
- **API Gateway (El Punto de Entrada Único):** En lugar de que tu frontend (React, Vue, etc.) o aplicación móvil tenga que conocer las 10 direcciones IP diferentes de tus 10 microservicios, solo conocerá una: <https://api.stresstarck.com>.

- **Ejemplos de llamadas y respuestas (fragmentos Postman o Swagger).**

Acceso a un Recurso Protegido (Éxito)

El estudiante (ya logueado) quiere ver sus cursos. El frontend envía el token access en la cabecera Authorization.

### **RESPONSE (200 OK):**

1. El API Gateway recibe la petición.
2. Válida la firma del token JWT (¡es válido!).
3. Entrega la petición al Servicio de Cursos.
4. El Servicio de Cursos devuelve la lista de cursos para ese estudiante

Acceso Sin Autenticación (Error 401)

Un usuario (o un atacante) intenta acceder a los cursos sin un token.

Headers: *(Sin cabecera Authorization)*

### **RESPONSE (401 Unauthorized):**

1. El API Gateway detiene la petición en seco. Ni siquiera llega al Servicio de Cursos.

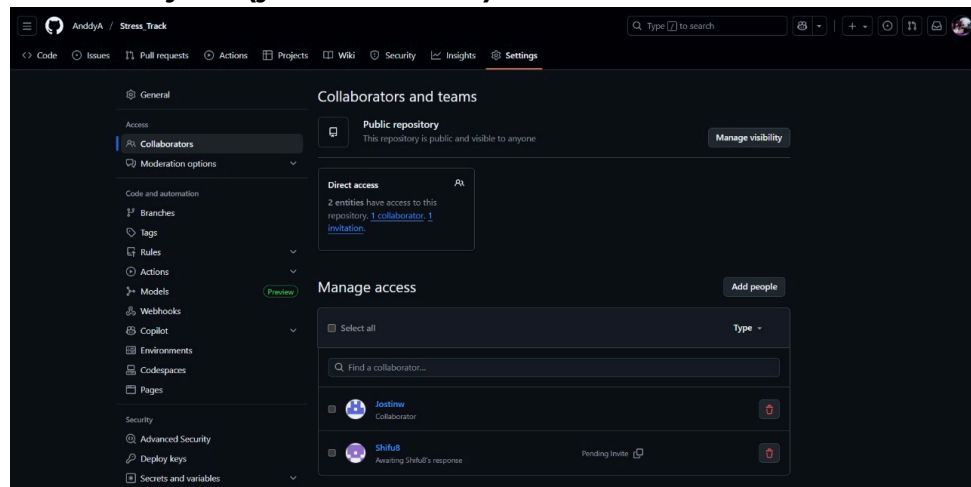
### 3. Herramientas colaborativas utilizadas

- Describir el uso de Trello, Taiga, GitHub Projects, Discord u otras herramientas para la coordinación del equipo.
- Evidenciar cómo estas herramientas contribuyeron a la planificación y seguimiento de tareas.

Para una gestión de proyecto eficiente y una coordinación efectiva, nuestro equipo centralizó el trabajo en GitHub Projects y utilizó Google Meet como principal canal de comunicación.

A continuación, se detalla cómo contribuyeron estas herramientas a la planificación y seguimiento de nuestras tareas:

#### GitHub Projects (y GitHub Issues)



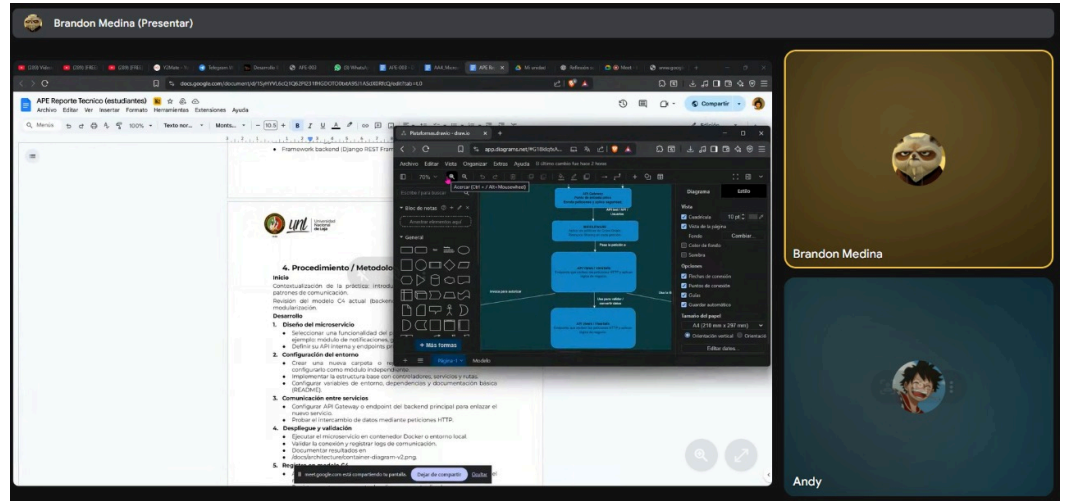
#### Descripción del uso:

GitHub Projects fue nuestro sistema de gestión de tareas, implementado con un tablero Kanban. Cada tarea, funcionalidad o error (bug) se documentó como un GitHub Issue, asignando responsables y etiquetas de prioridad.

#### Contribución a la Planificación y Seguimiento:

El uso del tablero permitió descomponer el proyecto en Issues manejables, priorizar el trabajo y realizar un seguimiento continuo del progreso moviendo las tarjetas entre columnas. Esta metodología brindó una trazabilidad completa y una visión en tiempo real del avance técnico del proyecto, facilitando la organización y la gestión eficiente de las tareas dentro del equipo.

## Coordinación y Comunicación: Google Meet



### Descripción del uso:

Utilizamos Google Meet como herramienta para la comunicación síncrona, enfocada en reuniones diarias de seguimiento (Daily Stand-ups) y sesiones de toma de decisiones.

Esto nos permitió compartir pantalla y resolver dudas al instante.

### Contribución a la Planificación y Seguimiento:

La coordinación del equipo se fortaleció mediante una comunicación inmediata que facilitó la resolución de bloqueos, garantizando una colaboración ágil. Además, las reuniones periódicas por Meet permitieron mantener a todos los integrantes alineados con el estado del Project Board de GitHub, identificando y abordando de forma proactiva cualquier posible retraso en el desarrollo del proyecto.

#### 4. Buenas prácticas y aprendizajes

- **Identificar dos buenas prácticas aplicadas durante el desarrollo.**

##### *1. Planificación y seguimiento con herramientas colaborativas*

Se utilizó GitHub Projects como tablero Kanban para organizar tareas, asignar responsabilidades y priorizar el trabajo. Complementariamente, se realizaron reuniones periódicas por Google Meet para sincronizar avances, resolver bloqueos y tomar decisiones técnicas.

Esta práctica permitió mantener una visión clara del progreso, asegurar la trazabilidad de las tareas y fortalecer la coordinación del equipo durante todo el desarrollo.

## 2. Control de versiones y flujo colaborativo en GitHub

Se aplicó una gestión de código basada en ramas (branching model) dentro del repositorio, utilizando branches para el desarrollo de nuevas funcionalidades, revisiones por pull requests y validación antes del merge a la rama principal (main).

Esta práctica fomentó la colaboración ordenada, permitió detectar errores antes de la integración y aseguró que el código en producción siempre se mantuviera estable y funcional.

- **Reflexión personal sobre cómo la modularización mejora la escalabilidad y mantenimiento del proyecto.**

La modularización del sistema a través de microservicios mejora significativamente la escalabilidad y el mantenimiento del proyecto.

Desde nuestra experiencia en este desarrollo, trabajar con módulos independientes permitió que cada parte del sistema pudiera evolucionar, desplegarse o corregirse sin afectar las demás, algo imposible en un monolito.

Por ejemplo, si en el futuro necesitamos cambiar el proveedor de correos o añadir una nueva funcionalidad de notificación, solo será necesario intervenir el microservicio correspondiente, sin alterar la lógica del sistema principal.

Esta independencia no solo reduce el riesgo de errores globales, sino que también facilita la colaboración en equipo, ya que varios desarrolladores pueden trabajar simultáneamente en diferentes microservicios sin generar conflictos.

En conclusión, la modularización no solo impulsa la eficiencia técnica, sino que también refleja principios éticos como la responsabilidad y la transparencia, al permitir una gestión más clara y controlada de cada componente del sistema.

### Forma de entrega en el EVA

- **Texto en línea:** URL del repositorio con la carpeta /docs/aa4\_microservicios.pdf.
- **Archivo adjunto:** PDF con el informe técnico final.
- **Nombre del archivo:** AA4\_Microservicios\_EquipoX.pdf.

### Recomendaciones

- Usa un lenguaje técnico claro y profesional.
- Incluye al menos un **diagrama C4 actualizado** (Container o Component).
- Las capturas o diagramas deben tener fuentes legibles y estar correctamente etiquetados.
- Todas las citas o fuentes deben ser referenciadas (APA o IEEE).
- El documento no debe exceder **4 páginas**.

### Preguntas de reflexión

### 1. ¿Qué ventajas observas en la integración mediante APIs REST respecto a un monolito tradicional?

La integración mediante APIs REST permite que los diferentes módulos del sistema se comuniquen de forma independiente, utilizando protocolos estándar y bien definidos. A diferencia de un monolito, donde todas las funcionalidades están acopladas en una sola aplicación, los microservicios pueden actualizarse, desplegarse o escalarse sin afectar al resto del sistema.

Esto nos facilita el mantenimiento, la reutilización de componentes y la escalabilidad horizontal. Además, REST nos ofrece una comunicación ligera basada en HTTP, lo que mejora la interoperabilidad entre servicios desarrollados incluso en distintos lenguajes o frameworks.

### 2. ¿Cómo aportan las herramientas colaborativas a la gestión técnica de los microservicios?

Las herramientas colaborativas, como GitHub Projects, Trello, Discord o Meet Google, resultan esenciales para organizar y coordinar las tareas del equipo. También permiten planificar actividades, asignar responsabilidades y hacer seguimiento del progreso en tiempo real, evitando confusiones o duplicación de trabajo.

En este proyecto, su uso ayudó a mantener una comunicación constante entre los integrantes, registrar los avances y documentar decisiones técnicas. En conjunto, estas herramientas fortalecen la transparencia, la responsabilidad y la eficiencia del equipo durante el desarrollo de los microservicios.

### 3. ¿Qué riesgos existen al distribuir un sistema en varios microservicios y cómo pueden mitigarse?

Uno de los principales riesgos al trabajar con microservicios es la complejidad en la comunicación entre servicios, lo que puede generar errores de integración o demoras por latencia en las llamadas. También existe el riesgo de dificultar el monitoreo y la gestión de dependencias si no se implementa una buena estructura de control.

Para mitigar estos problemas, es fundamental aplicar buenas prácticas de diseño, como el uso de un API Gateway, manejo adecuado de logs, pruebas automatizadas de integración y herramientas de monitoreo. Además, una documentación clara y actualizada contribuye a mantener la coherencia del sistema distribuido.

## Evaluación

| Criterio                                       | 2 – Logro alto                                                                     | 1 – Logro medio                                         | 0 – Bajo / Sin evidencia                  |
|------------------------------------------------|------------------------------------------------------------------------------------|---------------------------------------------------------|-------------------------------------------|
| <b>1. Descripción del microservicio</b>        | Describe con detalle el propósito, estructura y relación con el sistema principal. | Descripción parcial o sin relación con el sistema.      | No describe el microservicio.             |
| <b>2. Integración mediante APIs</b>            | Explica claramente la comunicación entre servicios con ejemplos y evidencias.      | Describe integración de forma superficial o incompleta. | No presenta integración ni evidencias.    |
| <b>3. Uso de herramientas colaborativas</b>    | Evidencia el uso activo y documentado de herramientas colaborativas.               | Menciona herramientas sin evidencias concretas.         | No menciona herramientas ni colaboración. |
| <b>4. Documentación y presentación técnica</b> | Informe bien estructurado, con diagramas y formato profesional.                    | Documento incompleto o con errores formales.            | Documento ausente o no técnico.           |
| <b>5. Reflexión crítica y buenas prácticas</b> | Reflexión profunda sobre ventajas, retos y aprendizajes del desarrollo modular.    | Reflexión breve o superficial.                          | No presenta reflexión.                    |

## Bibliografía sugerida

- Newman, S. (2021). *Building Microservices*. O'Reilly Media.
- Richardson, C. (2022). *Microservices Patterns*. Manning Publications.
- OWASP Foundation (2023). *OWASP Secure Microservices Design*.
- Docker Inc. *Docker Documentation*.
- GitHub Docs. *Collaborative Development Workflows*.